# Reflection: A Four-Agent Research Crew

UCLA Extension, Agentic AI, Week 1. CrewAI sequential crew over a 15-paper corpus.

## 1. How I designed the agents, and why

The crew is a sequential pipeline of four agents whose roles do not overlap: a **Research Query Strategist** (plans the search and never answers), a **Corpus Investigator** (the only agent with the `search_papers` tool), an **Evidence Synthesis Analyst** (finds themes, tensions and gaps, with no tools), and an **Academic Literature Review Writer** (produces the seven-section report). I gave each backstory a behavior it must show, not just a title. The investigator "treats the first result as a lead, never as an answer" and "only reports what the tool returned"; the synthesizer is "strict about provenance"; the writer "cites only papers in the citation key." Tasks have explicit output formats and use `context=` so each agent sees everything upstream (the writer sees the plan, the evidence log, and the synthesis).

**Tradeoffs I chose.** (a) Tool access for one agent only. This makes grounding auditable, since every claim traces to a tool call in the log and downstream agents cannot quietly fall back on memorized knowledge. The cost is that the synthesizer cannot ask for more evidence when it finds a gap. (b) CrewAI `memory` off: it defaults to OpenAI embeddings, which this Gemini setup does not configure, and task context already carries what each agent needs. (c) A citation key (label, ID, title) built from `paper_index.json` is injected into the tasks so agents can only cite real corpus papers. I considered a fifth "fact-checker" agent, but a deterministic check in code was cheaper and more reliable (section 3).

## 2. ReAct-style reasoning in the agent output

CrewAI uses native function calling, so the log shows the Action (tool call) and Observation (retrieved passages) but no free-text "Thought:" lines. The reasoning is visible in how each action changes in response to the previous observation. From the run that produced the final multi-agent report (`outputs/logs/final6_q2_verbose.log`, 25 searches, six papers retrieved):

```
A1 search("Wooldridge social ability communicate with other agents ...") → 6/6 Wooldridge
A10 search(wang_2023_survey "multi-agent" "cooperative" ...) → 0 Wang passages (4 Xi, 2 CAMEL)
A14 search("A Survey on Large Language Model based Autonomous Agents" "multi-agent") → 5/5 Wang
A17 search("send" "receive" "generate_reply" autogen) → 6/6 AutoGen
A18 search("GroupChat" "GroupChatManager" "speaker" autogen) → 6/6 AutoGen
A19 search("broadcast the selected speaker ...") → 5 AutoGen passages on speaker selection
```

**What this shows.** After A10 returned nothing from the Wang survey, the agent stopped using the paper's ID and used its exact title (A14), which retrieved it. The observation changed the next action. A17 to A19 show the agent drilling into one paper, moving from the message API to the group-chat manager to the speaker-selection passage. That is the mechanism the final report describes (a manager that selects the speaker and broadcasts the reply). The loop also has weaknesses. IDs in query text (A10, A16, A23) are metadata, not passage text, so they add noise, and the agent repeated the mistake after having seen it fail. Several queries returned the same paper repeatedly (A8 and A9 both returned only Generative Agents). A second feedback loop is the coverage guardrail. In an earlier run on a smaller model (`final5_q2_verbose.log`) it rejected the evidence log twice ("3 distinct papers and 10 excerpts; need at least 6 papers and 8 excerpts", then 4 papers) and the agent responded by searching for Wooldridge & Jennings and a survey, after which the log passed. The synthesizer and writer show reasoning only in their outputs, for example the tension the final report draws between centralized orchestration (AutoGen's group-chat manager) and emergent peer-to-peer coordination (Generative Agents).

## 3. What worked, what did not, and what I would change

**What did not work, in order of discovery.**

- **Provider failures.** The first run died on a Gemini `503` (model overloaded) after several minutes, and the provided `main.py` swallowed the error and exited with code 0 and no report. I added retry with backoff in `run_research`. The stronger free-tier models then hit a limit of 20 requests per day, so I ran on `gemini-3.5-flash-lite` until I bought API credits. The final reports use `gemini-3.8-flash`.
- **Fabricated references.** The writer produced plausible but wrong author lists (for example invented co-authors for the Wang survey). **Ambiguity:** two papers by "Yao et al., 2023" (ReAct and Tree of Thoughts) were cited identically. **Format drift:** the writer later cited by paper ID in backticks, or leaked ID suffixes (`2023_survey`).

- **Lazy retrieval on the small model.** The lite model made as few as 1 to 2 searches in some runs and missed CAMEL, the central multi-agent paper, in others. The stronger model made 25 searches per question.

**What worked.** The fixes for citations were code, not prompts. `citations.py` regenerates the References section from the paper index for exactly the papers cited, converts ID-style citations to author-year labels, disambiguates same-year papers (2023a/2023b), and warns on ambiguity. Prompt changes alone had not fixed either problem. A `guardrail` on the search task rejects thin evidence and names the papers not yet used. It fired on the small model and was not needed once the stronger model was used. Model strength mattered most for depth: the final reports contain numbered tensions, label inferences as "a plausible implication" without a citation, and state gaps the corpus leaves open.

**Remaining limits.** The reports cite 9 and 5 of 15 papers; the multi-agent report is well supported but narrow. I spot-checked key claims (group-chat speaker selection, forward error propagation, inception prompting, the safeguard agent) against the retrieved excerpts, but did not verify every sentence, and some inferences are stated more confidently than the text supports. With more time I would add an automated groundedness check that verifies each cited claim against the retrieved excerpt, and test whether giving the synthesizer a search tool improves coverage.

## 4. What production would require

- **Reliability and cost:** a paid-tier LLM with real quotas, retries with backoff (added), timeouts, per-run token and cost budgets, and caching of repeated queries. One run makes dozens of model calls, so cost scales with question volume.
- **Evaluation:** automated checks that every citation resolves to the index (done) and that every claim is supported by a retrieved passage; the provided retrieval metrics (Recall@k, MRR) tracked on a regression set so model or prompt changes cannot silently degrade quality.
- **Observability:** structured traces of every tool call and guardrail decision (the verbose log is a start) and alerting on failures. The silent exit-code-0 failure showed why this matters.
- **Safety and data:** treat retrieved text as untrusted (prompt injection through documents), keep API keys in a secrets store instead of a `.env` file, and re-embed on a schedule when the corpus changes.
- **Human review:** a reviewer approves any output that leaves the organization; the system's job is to make review fast by exposing sources.